

DER WICHTIGSTE ÜBERBLICK

Drei Wege, einen Agenten zu bauen

	AGENT BUILDER in M365 Copilot	ENTWICKLERWEG Manifest / Teams Toolkit	COPILOT STUDIO
Wer kann das	alle mit Copilot-Lizenz, ohne Code	Entwickler:innen	nur wenige Zugänge bei uns
Wissensquellen	ja — bis 20 Stück	ja	ja
Dateien erzeugen	ja — Code-Interpreter	ja	ja
Actions (schreiben, API rufen)	nein — heute nicht im Builder	ja — Connectoren, Web-API, Power Platform	ja
Trigger (läuft von selbst)	nein	eingeschränkt	ja
Für heute Nachmittag	das hier	nur mit Entwickler:in im Team	nicht verfügbar

Quelle: Microsoft Learn — „Agent Builder in Microsoft 365 Copilot“ und „Agents, Actions, and Connectors in the Microsoft 365 Ecosystem“, abgerufen 30.07.2026

Agent Builder: die Felder und ihre Grenzen

FELD	GRENZE	WOZU
Name	30 Zeichen	Anzeigenname
Beschreibung	1.000	damit Copilot weiß, wann er den Agenten nimmt
Instructions	8.000	das eigentliche Gehirn
Wissensquellen	20	SharePoint, Dateien, Websites
Starter-Prompts	beliebig	Einstiegsfragen für Nutzer

Was das für euch heißt

8.000 Zeichen sind **viel**. Unsere sechs fertigen Prompts im Repo brauchen 6.100 bis 6.800 Zeichen — ihr habt also noch gut 1.200 bis 1.900 Zeichen Platz für eure eigenen Regeln.

Antwortmodus wählbar: **Auto**, **Schnell** oder **Think deeper**. Für Prüf- und Analyse-Agenten lohnt „Think deeper“.

Code-Interpreter

Rechnet, analysiert Daten, erzeugt Diagramme und Dateien. **Kein** Zugriff nach außen, **keine** API-Aufrufe.

Bildgenerator

Erzeugt Bilder aus Prompts. Für die meisten Use Cases heute irrelevant — nicht davon ablenken lassen.

„Try it“-Tab

Testen, während ihr baut. Aktiv, sobald Name, Beschreibung und Instructions stehen. **Nutzt das früh.**

Quelle: Microsoft Learn — „Build agents with Agent Builder“, Stand 26.05.2026

DER HÄUFIGSTE STILLE BLOCKER

Woran euer Agent darf — und woran nicht

✓ Geht mit jeder Copilot-Lizenz

- SharePoint-Sites, -Ordner, einzelne Dateien
- hochgeladene Dateien
- öffentliche Websites

Zusätzlich mit M365-Copilot-Add-on

- eure Outlook-Mails
- Teams-Chatnachrichten
- Copilot-Connectoren, die im Tenant freigeschaltet sind

Klärt das in den ersten zehn Minuten

Wenn euer Use Case an einer Quelle hängt, an die ihr nicht drankommt, ist er tot — und zwar leise, ihr merkt es erst um 17 Uhr. **Prüft zuerst, ob ihr die Daten öffnen könnt.** Wenn nicht: Datei exportieren und direkt an den Agenten hängen. Das ist kein Kompromiss, das ist der übliche Weg.

Zum Bauen eines Agenten wird eine Microsoft-365-Copilot-Lizenz benötigt · Quelle: Microsoft Learn, Agent-Builder-Voraussetzungen

WAS DARAUS FOLGT

Der Agent legt vor, ein Mensch schickt ab



Warum das keine Notlösung ist

Der Agent macht die Arbeit, die weh tut: lesen, strukturieren, formulieren, Akzeptanzkriterien ableiten, Randfälle finden. Das Einfügen dauert 30 Sekunden.

Der Nutzen liegt zu 95 % vor dem letzten Klick.

Und es ist zukunftssicher

Wenn Actions kommen — im Entwicklerweg gibt es sie heute schon — schaltet ihr genau diesen einen Schritt durch. Instructions, Wissensquellen und Prüflogik bleiben unverändert. **Ihr baut nichts weg.**



So baut ihr in fünf Minuten einen Agenten

- 1 M365 Copilot öffnen** und auf **Neuer Agent** klicken
rechts in der Seitenleiste unter „Agents“
- 2 Beschreiben statt konfigurieren.** In normalem Deutsch sagen, was der Agent tun soll — Name, Beschreibung und Instructions füllen sich selbst oder „Skip to configure“ für die manuelle Variante, oder eine Vorlage nehmen
- 3 Auf „Configure“ wechseln** und die generierten Instructions durch euren eigenen Prompt ersetzen
aus dem Repo, Ordner prompts/
- 4 Wissensquellen hinzufügen** über das **+** — SharePoint-Ordner oder eine hochgeladene Datei
bis zu 20 Stück
- 5 Capabilities einschalten**, falls ihr Dateien oder Diagramme braucht
Schalter „Create documents, charts, and code“
- 6 Auf „Try it“ testen** — mit der Beispiel-Eingabe aus derselben Repo-Datei
hier merkt ihr in 30 Sekunden, ob der Prompt trägt
- 7 Nachschärfen und teilen.** Rechne mit drei bis vier Durchläufen, bis er sitzt
das ist normal und ist die eigentliche Arbeit

Wann lohnt sich überhaupt ein Agent?

Chatbot

Du fragst, es antwortet. Den nächsten Schritt machst du. Einmalige Frage, einmalige Antwort.

Agent

Du gibst ein **Ziel** und feste Regeln. Er zerlegt in Schritte, arbeitet sie ab, prüft sein Ergebnis gegen die Regeln — und ist **wiederverwendbar**: zwanzig Leute benutzen denselben.

Die zwei Bedingungen — beide müssen erfüllt sein

1. Mehrere Schritte. Bei einem Schritt reicht ein guter Prompt. Baut keinen Agenten drumherum, das kostet nur Zeit.

2. Prüfbares Ergebnis. Ihr müsst sagen können, woran man erkennt, dass die Ausgabe gut ist. Sonst kann sie niemand freigeben.

Wer bei Bedingung 2 hängt, holt einen Coach. Das ist der Punkt, an dem ein Use Case entweder scharf wird oder still scheitert.

DER HÄUFIGSTE FEHLER IST EIN ZU KURZER AUFTRAG

Vier Teile — und einer wird fast immer vergessen

TEIL	BEANTWORTET	BEISPIEL
ROLLE	Wer ist der Agent — und wofür nicht ?	„Du unterstützt eine Testerin. Du schreibst in kein System.“
AUFGABE	Was genau ist zu tun, in welcher Reihenfolge?	„Leite Testfälle ab: Happy Path, Fehler, Grenzwerte, Berechtigungen.“
REGELN	Was ist Pflicht, was verboten?	„Übernimm Feldnamen wörtlich. Erfinde keine Bezeichnungen.“
FORMAT	Wie sieht die Antwort aus?	„Tabelle mit genau diesen Spalten: ID, Titel, Vorbedingung, Schritte...“

Ohne FORMAT

kommt Fließtext zurück, und ihr sortiert das Ergebnis von Hand nach. Kostet mehr Zeit, als der Agent gespart hat.

Ohne REGELN

bewertet das Modell nach Gefühl — und zwar jedes Mal anders. Zwei Durchläufe, zwei Ergebnisse, kein Vertrauen.

DIE EINE ZEILE, DIE IN JEDEN PROMPT GEHÖRT

Warum Modelle Lücken füllen — und was dagegen hilft

Erfinde nichts. Fehlt eine Angabe, schreib FEHLT und formuliere die Rückfrage.

Ohne diesen Satz

Ein Modell antwortet lieber plausibel als gar nicht. Es erfindet einen Feldnamen, eine Frist, eine Zuständigkeit — und **man sieht dem Ergebnis nicht an, was echt war**. Das ist der gefährlichste Fehlermodus, weil das Ergebnis gut aussieht.

Mit diesem Satz

Ihr bekommt statt einer erfundenen Antwort eine ehrliche Lücke — und eine Rückfrage, die ihr weiterleiten könnt. **Ein Ergebnis mit drei FEHLT-Markern ist besser als eines ohne.**

In einer Bank ist das der Unterschied zwischen brauchbar und gefährlich. Der Satz steht in allen sechs Prompts im Repo drin.

Beispiel 1 — gebaut von einem Kollegen, ohne eine Zeile Code

Das Problem

Der Kundenservice schreibt Englisch, unsere Kundinnen und Kunden lesen Deutsch. Bisher übersetzt jemand von Hand und hofft, dass die Bankbegriffe stimmen.

Was der Agent tut

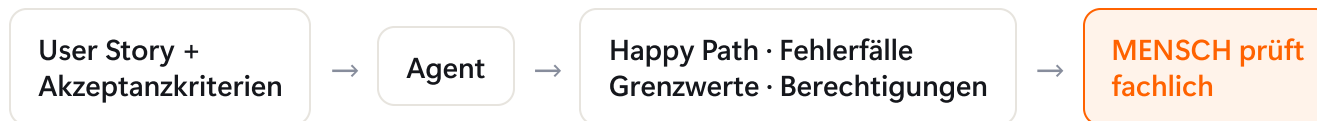
Übersetzt in beide Richtungen, erkennt die Richtung selbst, zieht Fachbegriffe aus einem angehängten Glossar und lässt Beträge, IDs und Zuständigkeiten unangetastet.

Worauf ihr schauen sollt

Unter der Übersetzung steht ein **Audit-Log**: welcher Begriff kam aus dem Glossar, welcher wurde nicht gefunden, in welche Richtung wurde übersetzt.

Der Punkt: Nicht die Übersetzung ist das Kunststück, das kann jedes Modell. Das Kunststück ist, dass ein Mensch in fünf Sekunden prüfen kann, ob er dem Ergebnis trauen darf. Genau das meinte ich mit „prüfbares Ergebnis“ — **und es kostet drei Zeilen Prompt.**

Beispiel 2 — und die Stelle, an der es nicht stimmt



Was der Agent zuverlässig liefert

Die Struktur und die Vollständigkeit über alle vier Kategorien. Grenzwerte und Berechtigungsfälle werden unter Zeitdruck als Erstes vergessen — der Agent vergisst sie nie, weil sie in der Aufgabe stehen.

Was ihr prüfen müsst

Die **fachliche** Richtigkeit. Der Agent kennt eure Produktlogik nicht. Er erzeugt einen plausibel klingenden Testfall auch dort, wo die Anforderung unscharf war — **das seht ihr gleich live.**

WEIL VIER DER ELF USE CASES DARAN HÄNGEN

Azure DevOps anbinden — drei Stufen

STUFE	WAS IHR KÖNNT	WIE	AUFWAND
1 • ohne Technik	Browser bedienen	Query bauen, Work-Item-Text kopieren, Agenten-Ausgabe von Hand als Ticket anlegen	0 Minuten Setup
2 • Excel / CSV	Dateien exportieren	Query als CSV, Agent füllt Spalten, Re-Import — ID -Spalte leer lassen heißt „neu anlegen“	10 Minuten
3 • Skript	node benutzen	REST API mit eurem eigenen Login — kein Personal Access Token	15–30 Minuten

Stufe 3 ist heute getestet

Vier Skripte im Repo, gegen unsere echte Instanz geprüft: Anmeldung, Work Item lesen, Abfrage über 47 Test Cases, Kommentar schreiben. Immer erst `--dry-run` — das druckt den Request und sendet nichts.

Die Ansage für heute

Für eine Demo reicht **Stufe 1 oder 2**. Ihr verliert dabei keinen Punkt bei der Bewertung. Ein Team, das um 18 Uhr noch an einer Anmeldung scheitert, hat den Tag verloren.

WAS EUCH HEUTE WIRKLICH AUFHÄLT

Zwei Sackgassen — und wie ihr sie in fünf Minuten umbaut

1 · Die Quelle, an die ihr nicht drankommt

Symptom: Der Agent soll etwas auswerten, das in einem System liegt, für das ihr keine Freigabe habt.

Umbau: Daten einmal exportieren und als Datei an den Agenten hängen. Für eine Demo völlig ausreichend.

2 · Der Use Case, der schreiben will

Symptom: „...und dann legt er das Ticket an.“ Der Agent Builder kann das heute nicht.

Umbau: Der Agent erzeugt den fertigen Text, ein Mensch fügt ihn ein. Der Nutzen bleibt fast vollständig erhalten.

Beides kostet fünf Minuten — um 16 Uhr

Um halb sieben nicht mehr. Wenn einer der beiden Sätze auf euch zutrifft, **holt mich sofort**, nicht am Ende. Ich bin in Frankfurt vor Ort, für Nürnberg im Chat.



BILDSCHIRM WECHSELN → Tab 4 · das Repo kurz zeigen danach zurück, letzte Folie stehen lassen

Alles in einem Repo — kein Git nötig

github.com/Wldefant/ai-agent-hackathon-starter

Draufklicken, Text kopieren, in den Agenten einfügen. Wer mit Git arbeitet, klonet. Auf jedem Tisch liegt eine Karte mit dem Link.

6 fertige Prompts

je mit Beispiel-Eingabe, erwarteter Ausgabe und Prüfschritt

Azure DevOps

drei Stufen, Skripte gegen die echte Instanz getestet

3 Copilot-Beispiele

für die Teams mit Entwickler:in, je 2 Minuten

Ein echtes 6-Agenten-System

aus einem anderen Team — zum Abschauen, nicht zum Nachbauen